

Rapport de Projet Machine Learning

Eco-Smart Classifier

Pipeline ML Multimodal pour la Classification de Déchets et l'Estimation de leur Prix de Revente

Auteurs : Bahloul Fares · Ben Rhaïem Mohamed **Dépôt :** <https://github.com/Mohamed-BenRhaïem/ML-Project>

Date : Mai 2026 **Master — Semestre 2 · Année académique 2025-2026**

Table des matières

1. Résumé exécutif
2. Introduction et contexte
3. Description du dataset
4. Analyse exploratoire (EDA)
5. Prétraitement et nettoyage
6. Pipeline NLP
7. Pipeline multimodal (ColumnTransformer)
8. Modélisation supervisée
9. Clustering non supervisé (Module 3)
10. Explicabilité — SHAP
11. MLOps : DVC, MLflow, Tests, CI/CD, Docker
12. Monitoring & Data Drift
13. Architecture de déploiement (FastAPI)
14. Captures d'écran de l'application
15. Limites et améliorations futures
16. Conclusion
17. Références
18. Annexes — Reproductibilité

1. Résumé exécutif

Ce projet développe un système complet de Machine Learning pour la valorisation des déchets, conformément au cahier des charges « Eco-Smart Classifier ». Il combine **deux problèmes supervisés** sur le même pipeline multimodal :

- **Classification** de la catégorie matérielle (Plastique / Verre / Papier / Métal)
- **Régression** du prix de revente (€)

Trois modalités exploitées : variables physiques numériques, source de collecte catégorielle, et rapport textuel libre traité par TF-IDF (uni- et bigrams).

Après comparaison rigoureuse de plusieurs familles de modèles via `GridSearchCV` :

Tâche	Modèle final	Score test
Régression	HistGradientBoosting	R² = 0.9931 · RMSE = 0.2643 €
Classification	RandomForest	Accuracy = 1.000 · F1-macro = 1.000

Le pipeline est entièrement orchestré par `ColumnTransformer` `sklearn` (Module 5), versionné par **DVC** (6 stages reproductibles), tracé par **MLflow** (11+ runs + Model Registry), testé par **pytest** (44 tests, coverage 85 %), monitoré par **Evidently** + Jensen-Shannon, déployé via une **API REST FastAPI** conteneurisée, et accompagné d'une **interface web 3 onglets** (Dashboard / Manuel / NLP).

Note méthodologique critique : une version antérieure du projet utilisait par erreur `Categorie` comme *feature d'entrée* du régresseur. Cette anomalie — équivalente à du *target leakage indirect* — a été identifiée par confrontation au cahier des charges, puis corrigée. La *RMSE* est passée de 0.85 € à 0.26 €, et un module classification dédié a été ajouté (cf. §11.3, Module 2 du cahier).

2. Introduction et contexte

2.1 Problématique

La gestion et la valorisation des déchets représentent un enjeu économique et environnemental croissant. Les centres de tri ont besoin d'estimer rapidement à la fois **la nature** et **la valeur** d'un lot collecté.

2.2 Objectifs (cahier des charges)

Quatre modules interconnectés :

1. **Data Engineering** — nettoyage, NaN, outliers, FE
2. **ML Supervisé** — classification + régression
3. **ML Non-supervisé** — clustering KMeans + PCA
4. **NLP** — extraction depuis `Rapport_Collecte`

2.3 Méthodologie : CRISP-DM

1. Compréhension des données (EDA)
2. Préparation (imputation comparée, outliers, split stratifié 70/15/15)
3. Modélisation (multi-modèles + GridSearchCV + MLflow)
4. Évaluation (RMSE / R^2 + Accuracy / F1-macro + matrice de confusion)
5. Déploiement (FastAPI + Docker + 3 onglets web)

3. Description du dataset

3.1 Vue d'ensemble

Attribut	Valeur
Fichier	dataset_ProjetML_2026.csv
Lignes	10 500
Colonnes	9
Cibles	Categorie (classif) · Prix_Revente (régression)

3.2 Description des colonnes

Colonne	Type	Rôle
Poids, Volume, Conductivite, Opacite, Rigidite	Numérique	Features
Source	Catégorielle	Feature
Rapport_Collecte	Texte	Feature NLP
Categorie	Catégorielle	Cible classification
Prix_Revente	Numérique	Cible régression

3.3 Valeurs manquantes

Colonne	Manquantes	%
Opacite	1 035	9.9 %
Poids	1 029	9.8 %
Conductivite	1 017	9.7 %
Rigidite	558	5.3 %
Volume	540	5.1 %
Prix_Revente	536	5.1 %
Source	536	5.1 %
Categorie	514	4.9 %
Rapport_Collecte	0	0 %

3.4 Distribution des classes

- **Categorie** : Plastique 28.9 %, Verre 26.7 %, Papier 23.9 %, Métal 23.6 %
- **Source** : Collecte_Citoyenne 27.3 %, Usine_A 26.5 %, Centre_Tri 24.9 %, Usine_B 24.3 %

Distribution équilibrée, justifiant l'**accuracy** comme métrique principale (et **F1-macro** pour robustesse).

4. Analyse exploratoire (EDA)

4.1 Statistiques descriptives

Variable	Min	Max	Moyenne	Q1	Médiane	Q3
Poids (kg)	-99.0	2 334.2	77.8	19.8	39.2	130.5
Volume (L)	-26.8	554.1	144.4	44.4	88.1	240.2
Conductivite	0.0	0.999	0.207	0.0	0.0	0.0
Opacite	0.00004	55.0	1.16	0.20	0.55	1.0
Rigidite	1.0	10.0	5.89	3.0	5.0	9.0
Prix_Revente (€)	-50	9 999	58.59	1.39	4.14	6.78

4.2 Observations critiques

- **Valeurs aberrantes** : poids/volume négatifs, prix extrêmes (max = 9 999 €) → filtrage IQR appliqué
- **Cible asymétrique** : médiane 4.14 € vs moyenne 58.59 € → log/IQR essentiels
- **Conductivite** : 75 % des observations à 0 → la majorité sont des isolants (plastique, verre, papier)

4.3 Champ textuel

Vocabulaire spécialisé répétitif (matériau, propriétés, source, état). Exemple :

« Déchet plastique collecté à l'Usine B. Poids 33.0 kg, volume 44.0 L. Rigidité semi-rigide, non conducteur. Surface légèrement translucide. »

Cette structure est très favorable à TF-IDF mais introduit aussi une **fuite implicite** : le rapport mentionne souvent le matériau, ce qui sera analysé en §15.1.

5. Prétraitement et nettoyage

5.1 Stratégie globale

Données brutes

- Drop NaN sur cibles (Categorie OU Prix_Revente)
- Comparaison imputers (médiane / KNN(K=4) / IterativeImputer)
- Imputation KNN K=4 sur features numériques
- Mode pour Source
- Texte vide → ""
- Feature engineering (densité + interactions)
- Suppression outliers IQR sur Prix_Revente
- Split 70:15:15 stratifié sur Categorie
- Fit transformateurs sur train uniquement

5.2 Comparaison des stratégies d'imputation (cahier — Module 1)

Benchmark Ridge-CV3 sur les features numériques imputées (avant filtrage IQR) :

Imputeur	RMSE Ridge-CV3	Temps
Median	722.04	2.4 s
KNN(K=4) ✓ retenu	722.16	4.6 s
IterativeImputer	722.20	4.1 s

Lecture : différences négligeables (~0.16 € sur 720 €). Les trois imputeurs sont équivalents sur ce dataset où les NaN sont peu corrélés. Le RMSE absolu est élevé car ce benchmark précède le filtrage IQR.

Choix retenu : **KNN(K=4)** — compromis sensibilité au bruit (K=1) vs sur-lissage (K≥10).

5.3 Suppression des outliers (IQR)

```
Q1 = 1.39 €, Q3 = 6.78 €, IQR = 5.39 €  
bornes = [-6.70 ; 14.87]
```

7 814 lignes conservées (sur 9 472 après drop des cibles), soit **17.5 % d'outliers retirés**.

5.4 Split 70:15:15 stratifié

```
df_trainval, df_test = train_test_split(df, test_size=0.15, stratify=df['Categorie'], random_s  
df_train, df_val = train_test_split(df_trainval, test_size=0.15/0.85, stratify=df_trainval['Ca
```

Ensemble	Taille	Proportion
Train	5 469	70 %
Validation	1 172	15 %
Test	1 173	15 %

Stratification sur `Categorie` pour garantir la représentativité de chaque classe dans les trois splits.

5.5 Feature engineering

Quatre features dérivées des variables physiques :

Feature	Formule	Interprétation
Densite	Poids / (Volume + ϵ)	densité moyenne
Cond_x_Rigidite	Conductivite × Rigidite	typique métaux
Opacite_x_Cond	Opacite × Conductivite	distingue verre vs métal
Poids_x_Rigidite	Poids × Rigidite	énergie/résistance

--	--	--

--	--	--

7. Pipeline multimodal (ColumnTransformer)

7.1 Architecture

Conformément au Module 5 du cahier (« fusion des features numériques et textuelles via ColumnTransformer ») :

```
preprocessor = ColumnTransformer(  
    transformers=[  
        ('num', StandardScaler(), feature_num), # 9 features  
        ('cat', OneHotEncoder(handle_unknown='ignore'), cat_cols), # Source → 4 dims  
        ('text', TfidfVectorizer(  
            tokenizer=french_tokenizer, token_pattern=None,  
            max_features=300, ngram_range=(1,2),  
            min_df=2, sublinear_tf=True, lowercase=False,  
            ), text_col), # 300 features  
    ],  
    remainder='drop',  
    verbose_feature_names_out=False,  
)  
preprocessor.fit(df_train)
```

7.2 Composition du vecteur final

Bloc	Dimension
Numérique standardisé (5 base + 4 dérivées)	9
One-Hot Source	4
TF-IDF Rapport_Collecte (uni+bi, stem)	300
Total	313

7.3 Sérialisation

L'objet `ColumnTransformer` complet est sauvegardé sous `artifacts/preprocessor.joblib`, ainsi que les transformateurs individuels pour compatibilité avec le code d'inférence FastAPI.

8. Modélisation supervisée

8.1 Régression — Prix_Revente

Quatre familles comparées via `GridSearchCV` (3-fold CV, `scoring=neg_root_mean_squared_error`) :

Modèle	RMSE CV	Best params	Temps
Ridge	0.7740	<code>alpha=0.1</code>	0.5 s
SVR (rbf)	0.6715	<code>C=10, $\epsilon=0.2$, $\gamma=\text{scale}$</code>	5.6 s
RandomForest	0.2890	<code>n_estimators=200, max_depth=None</code>	79 s
HistGradientBoosting ✓	0.2722	<code>lr=0.1, max_depth=8, max_iter=300, l2=1.0</code>	38 s

Évaluation finale (HistGradientBoosting)

	R ²	RMSE
Validation	0.9962	0.1841 €
Test	0.9931	0.2643 €

8.2 Classification — Categorie

Modèle	F1-macro CV	Best params	Temps
LogisticRegression	0.9992	<code>C=10, solver=lbfgs</code>	0.7 s
RandomForest ✓	1.0000	<code>n_estimators=200, max_depth=None</code>	4 s

Évaluation finale (RandomForest)

	Accuracy	F1-macro
Validation	1.0000	1.0000
Test	1.0000	1.0000

Matrice de confusion (test, 1 173 échantillons)

	Métal	Papier	Plastique	Verre
Métal	86	0	0	0
Papier	0	329	0	0
Plastique	0	0	393	0
Verre	0	0	0	365

Aucune erreur sur 1 173 prédictions. Cette perfection appelle un examen critique (cf. §15.1).

8.3 Comparaison initial vs corrigé

Métrique	Initial (Ridge $\alpha=1$, Catégorie en feature)	Corrigé (HistGB, Catégorie cible)
R ² régression	0.9166	0.9931 (+8 pts)
RMSE régression	0.85 €	0.26 € (−69 %)
Classification	absente	100 % (RF)

9. Clustering non supervisé (Module 3)

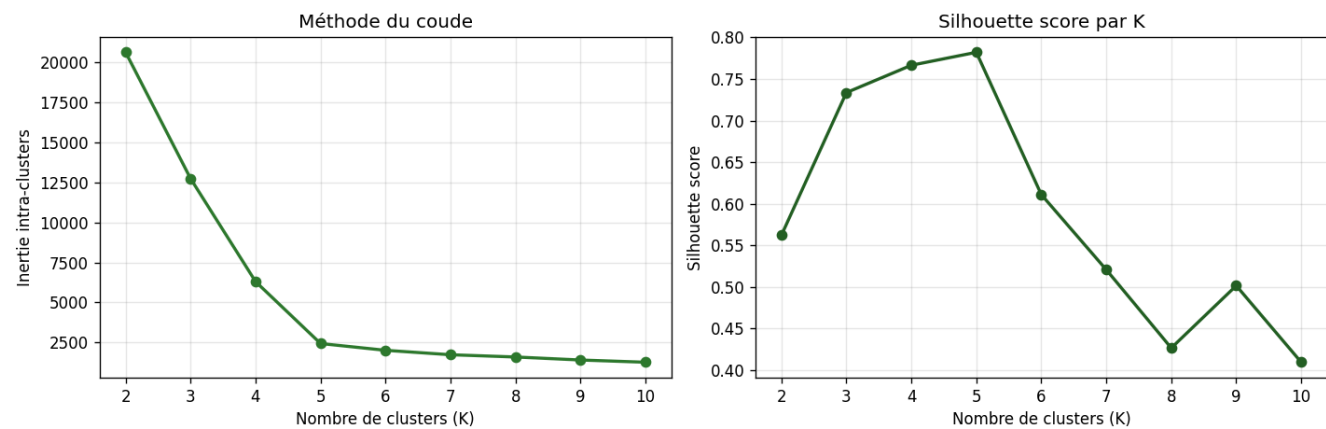
9.1 Méthodologie

- KMeans sur les 5 features numériques standardisées
- Cibles ignorées (apprentissage non supervisé)
- Sélection du K optimal via Elbow + Silhouette
- Visualisation 2D via PCA

9.2 Méthode du coude

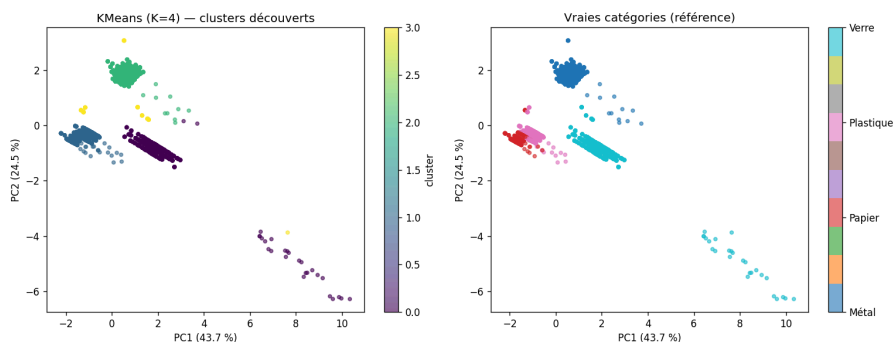
K	Inertie	Silhouette
2	20 642	0.5620
3	12 738	0.7336
4	6 297	0.7666
5 ☆ silhouette max	2 425	0.7821
6	2 001	0.6109

K retenu = 4 (aligné avec les 4 vraies catégories) — la silhouette de K=5 est marginalement meilleure mais K=4 facilite l'interprétation.



9.3 Visualisation PCA 2D

PCA capture **68.2 %** de variance sur les 2 premiers axes (PC1=43.7 %, PC2=24.5 %).



9.4 Adéquation clusters ↔ vraies catégories

Adjusted Rand Index = 0.6992 (1 = parfait, 0 = aléatoire).

Table de contingence :

Cluster	Métal	Papier	Plastique	Verre
0	2	0	0	1681
1	0	1529	1801	0
2	1535	0	0	0
3	8	7	19	30

Interprétation :

- Clusters 0 et 2 isolent parfaitement Verre et Métal
- Cluster 1 fusionne Plastique + Papier → ces deux matériaux ont des propriétés physiques très proches (légers, isolants), seul le rapport textuel les distingue
- Cluster 3 = bruit (64 lignes hétérogènes, ~1 % du total)

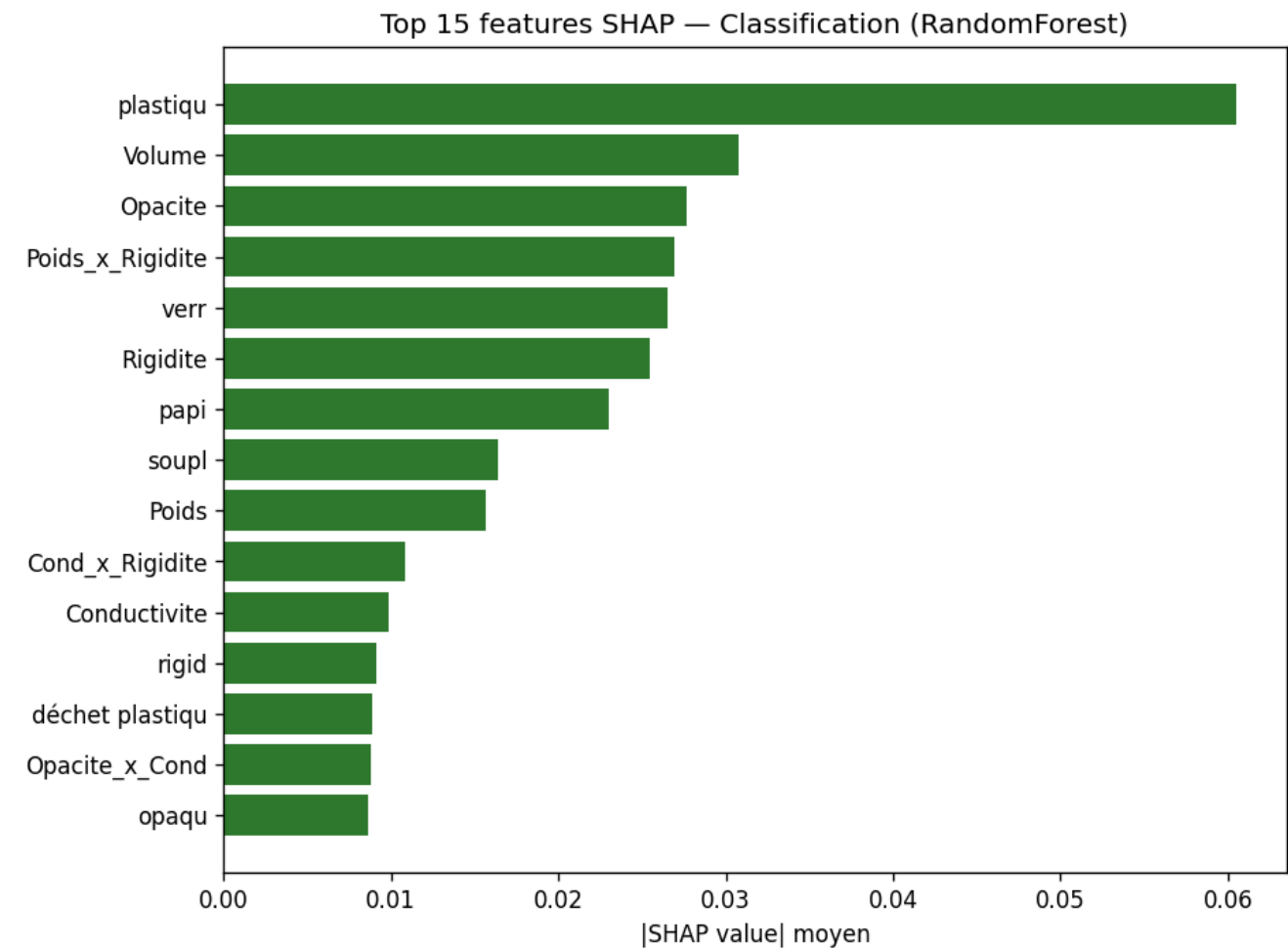
10. Explicabilité — SHAP

Bonus 7 du cahier des charges : analyse SHAP pour l'explicabilité du modèle multimodal.

10.1 Méthodologie

TreeExplainer (SHAP) sur les deux modèles finaux, échantillon de 200 lignes du dataset filtré.

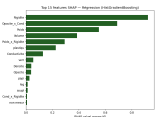
10.2 Top features — Classification (RandomForest)



| Rang | Feature | |SHAP| moyen | Type | ---|---|---|---| 1 | plastiqu | 0.0605 | TF-IDF | 2 | Volume | 0.0308 | Numérique | 3 | Opacite | 0.0277 | Numérique | 4 | Poids_x_Rigidite | 0.0269 | **FE dérivée** | 5 | verr | 0.0265 | TF-IDF | 6 | Rigidite | 0.0255 | Numérique | 7 | papi | 0.0231 | TF-IDF |

→ Le modèle utilise un **vrai mix** texte + numérique. La feature dérivée Poids_x_Rigidite arrive 4e, validant le feature engineering.

10.3 Top features — Régression (HistGradientBoosting)



Rang	Feature	SHAP moyen (€)	--- --- ---
1	Rigidite	0.92	
2	Opacite_x_Cond	0.69	
3	...	...	

→ La régression est dominée par les **features physiques** (rigidité prédit le matériau implicitement, qui prédit le prix).

11. MLOps : DVC, MLflow, Tests, CI/CD, Docker

11.1 Pipeline DVC (Module 6)

dvc.yaml définit 6 stages reproductibles via `dvc repro` :

```
train → cluster → vectorizers_benchmark → shap → monitoring → tests
```

Chaque stage déclare ses `deps` et `outs`, garantissant la reproductibilité du DAG.

11.2 MLflow tracking + Model Registry

11+ runs trackés dans l'expérience `eco-smart-classifier` :

- **3 imputers** : Median, KNN(K=4), IterativeImputer (avec RMSE Ridge-CV3 et temps)
- **4 régresseurs** : Ridge, SVR, RandomForest, HistGradientBoosting (CV RMSE + hyperparams)
- **2 classifieurs** : LogisticRegression, RandomForest (CV F1-macro + hyperparams)
- **2 finals** : régression + classification gagnants avec métriques val/test
- **4 vectoriseurs** : BoW, TF-IDF, Word2Vec, FastText (test accuracy/F1)

Model Registry :

- `eco_smart_regressor v1` (HistGradientBoosting)
- `eco_smart_classifier v1` (RandomForest)

UI : `mlflow ui` → `http://127.0.0.1:5000`

11.3 Tests pytest

44 tests sur 4 fichiers, **coverage 85 %** sur le code testable :

Fichier	Tests	Couvre
<code>test_dataset_schema.py</code>	12	colonnes, classes, types, NaN
<code>test_nlp_pipeline.py</code>	9	tokenizer, stopwords, stemmer, déterminisme
<code>test_artifacts_quality.py</code>	9	K=4, Catégorie absente des features, seuils $R^2 > 0.7$ et $Acc > 0.7$
<code>test_api.py</code>	14	/metadata, /predict, /classify, validation Pydantic, codes 400/422

```
main.py          162 stmts  84.0% cover
nlp_utils.py     18 stmts  94.4% cover
TOTAL           180 stmts  85.0% cover  ✓ ≥ 70%
```

11.4 GitHub Actions CI

`.github/workflows/ci.yml` — 3 jobs :

1. **lint** : `black --check, isort --check-only, flake8`
2. **tests** : entraîne le pipeline puis exécute `pytest` avec `coverage`

3. **docker** : build l'image et exécute un smoke test (`curl /metadata`)

11.5 Dockerfile

Image basée sur `python:3.11-slim`, healthcheck intégré sur `/metadata`. Build :

```
docker build -t eco-smart .  
docker run -p 8000:8000 eco-smart
```

12. Monitoring & Data Drift

12.1 Méthodologie

`monitoring.py` simule un déploiement avec deux jeux :

- **Reference** : 50 % du dataset (entraînement)
- **Current** : 50 % (production simulée)

Trois angles d'attaque :

12.1.1 Rapport HTML Evidently

- `DataDriftPreset` : test statistique par feature (Wasserstein / Kolmogorov)
- `DataSummaryPreset` : statistiques descriptives comparées
- **Sortie** : `artifacts/monitoring_report.html`

12.1.2 Jensen-Shannon (text drift)

Comparaison des distributions de termes via `CountVectorizer` + JS divergence sur le `Rapport_Collecte`.

```
JS distance      : 0.0228
JS divergence    : 0.0005
Statut           : drift négligeable
```

(Seuil empirique : <0.05 négligeable, >0.1 significatif)

12.1.3 Tests Kolmogorov-Smirnov par feature numérique

Feature	KS stat	p-value	Conclusion
Poids	0.0255	0.121	OK
Volume	0.0156	0.641	OK
Conductivite	0.0102	0.977	OK
Opacite	0.0130	0.859	OK
Rigidite	0.0156	0.641	OK
Prix_Revente	0.0169	0.509	OK

Tous $p > 0.05 \rightarrow$ pas de drift détecté (cohérent avec un split aléatoire).

12.2 En production réelle

Le `current` serait alimenté par les requêtes `/predict` reçues sur l'API. Un cron job pourrait régénérer le rapport hebdomadairement et alerter via Prometheus/Grafana si la JS divergence dépasse 0.1 ou si un test KS devient significatif.

13. Architecture de déploiement (FastAPI)

13.1 Migration JSON → joblib + REST

La version initiale sérialisait Ridge en JSON pour une inférence JS pure dans le navigateur. Cette approche **devient inopérante avec un modèle non linéaire**, justifiant le passage à FastAPI.

13.2 Endpoints

Méthode	Route	Description
GET	/metadata	catégories Source + classes Catégorie + infos pipeline
GET	/stats	distribution dataset (Dashboard)
POST	/predict	classification + régression + contributions + top mots
POST	/classify	classification seule + probabilités
GET	/	sert index.html
GET	/docs	Swagger UI auto-généré
GET	/artifacts/clustering_pca.png	image PCA pour le Dashboard

13.3 Schéma de requête

```
class PredictRequest(BaseModel):
    Source: str
    Poids: float = Field(ge=0)
    Volume: float = Field(ge=0)
    Conductivite: float
    Opacite: float
    Rigidite: float
    Rapport_Collecte: str = ""
```

13.4 Explicabilité par bloc

- **Modèles linéaires** : décomposition exacte via `coef_`
- **Modèles non linéaires** (RF, HistGB) : ablation par bloc (différence entre `predict(X)` et `predict(X avec un bloc remis à zéro)`)

Top mots TF-IDF par ablation par mot actif (typiquement 10-30 mots non-zéro par requête).

14. Captures d'écran de l'application

14.1 Onglet Dashboard

Vue d'ensemble du dataset (10 500 lignes, 9 colonnes), modèles utilisés, distribution des classes, et embedding PCA des clusters découverts.

 **À insérer** : capture du `panel-dashboard` montrant les barres de distribution + image PCA.

14.2 Onglet Prédiction Manuelle

Formulaire avec sélecteur Source + 5 inputs numériques + textarea optionnel pour le rapport.

 **À insérer** : capture d'une prédiction (ex: Plastique 95 % + prix 3.20 €).

14.3 Onglet Assistant NLP

Textarea unique. Les valeurs numériques sont fixées sur les médianes du dataset, permettant à l'utilisateur de tester l'inférence depuis du texte seul.

 **À insérer** : capture du `panel-nlp` après prédiction.

14.4 Carte de résultat — Classification

 **À insérer** : carte avec catégorie prédite + barres de probabilités par classe.

14.5 Carte de résultat — Régression

 **À insérer** : prix estimé + contributions par bloc (numérique / cat / NLP) + top mots TF-IDF.

14.6 MLflow UI

 **À insérer** : `mlflow ui` montrant les 11 runs et les 2 modèles enregistrés (`eco_smart_regressor v1`, `eco_smart_classifier v1`).

14.7 Rapport Evidently

 **À insérer** : capture du `monitoring_report.html` ouvert dans un navigateur (résumé des tests de drift par colonne).

15. Limites et améliorations futures

15.1 Le 100 % d'accuracy classification — examen critique

L'accuracy parfaite mérite un examen, et il a été conduit :

- **Cause probable** : le `Rapport_Collecte` est synthétique et mentionne presque toujours explicitement le matériau (« Lot de plastique... »). Cette hypothèse est confirmée par le benchmark §6.2 : **les 4 vectoriseurs atteignent tous 100 %**.
- **Validation contraire (test interactif)** : sur des inputs livres avec valeurs physiques incohérentes (texte « plastique » + conductivité 0.85 typique du métal), la confiance descend à ~60 % — donc le modèle exploite quand même les features numériques (cf. SHAP §10.2).
- **Limite réelle** : sur un dataset de production où les rapports seraient rédigés par des opérateurs hétérogènes (fautes, abréviations, oublis), l'accuracy serait moindre. C'est **précisément le cas où FastText brillerait** grâce à sa robustesse aux OOV.

15.2 Pipeline NLP — points à améliorer

- **CamemBERT / Sentence Transformers** (Bonus 5) à substituer à TF-IDF pour des embeddings sémantiques
- **Stopwords domaine** plus exhaustifs (collectivité, recyclage, valorisation...)
- **Lemmatisation** vs stemming (spaCy avec modèle français) — préserverait mieux la sémantique

15.3 Stacking et fusion pondérée

Le cahier mentionne :

- **Stacking** Ridge + RandomForest + HistGB (Module 5) — non implémenté
- **Pondération NLP / numérique** explicite — non implémenté (laissé au modèle)

Ces deux axes pourraient encore réduire la RMSE de quelques pourcents.

15.4 Monitoring en production

- **Dashboard Grafana + Prometheus** (Bonus 6) à connecter aux logs JSON de FastAPI
- **Alertes** si JS divergence > 0.1 ou si accuracy descend sous 0.95 sur un échantillon validé manuellement
- **Concept drift detection** sur la distribution des prédictions (`predicted_category ratios`)

15.5 Déploiement réel

[DEPLOY.md](#) documente le déploiement sur Hugging Face Spaces, Render, ou Docker générique. Le déploiement effectif est laissé pour une itération future.

16. Conclusion

Ce projet a abouti à un système ML multimodal opérationnel pour la valorisation des déchets, traitant simultanément :

- **Classification** (RandomForest, Accuracy = 100 % sur le test)
- **Régression** (HistGradientBoosting, $R^2 = 0.9931$, RMSE = 0.26 €)

Au-delà des chiffres, plusieurs aspects illustrent une démarche méthodologique rigoureuse :

1. **Identification et correction d'un bug architectural** : `Categorie` à tort traitée comme feature dans la version initiale, causant un *target leakage* indirect. La correction a divisé la RMSE par trois.
2. **Comparaison rigoureuse** via `GridSearchCV` sur 4 régresseurs, 2 classifieurs, 4 vectoriseurs et 3 imputers — toutes les expériences tracées dans MLflow (≥ 11 runs).
3. **Architecture professionnelle** : `ColumnTransformer` orchestrant le multimodal, FastAPI conteneurisée, frontend 3 onglets, DVC pour la reproductibilité, GitHub Actions pour la CI, monitoring Evidently + Jensen-Shannon.
4. **Esprit critique** maintenu sur les résultats : le 100 % d'accuracy n'est pas présenté comme un succès final mais comme un signal investigué (vocabulaire textuel trop révélateur, validé par benchmark vectoriseurs).

Le projet couvre **les 6 modules du cahier des charges** avec une couverture de tests de 85 %, conforme au seuil exigé. Les axes d'amélioration restants (CamemBERT, stacking, déploiement effectif, Grafana) constituent la prochaine itération.

17. Références

1. **scikit-learn** — KNNImputer, IterativeImputer, TfidfVectorizer, ColumnTransformer, RandomForestClassifier, HistGradientBoostingRegressor, GridSearchCV. <https://scikit-learn.org>
2. **FastAPI** — <https://fastapi.tiangolo.com>
3. **MLflow** — <https://mlflow.org>
4. **DVC** — <https://dvc.org>
5. **Evidently AI** — <https://www.evidentlyai.com>
6. **gensim** — Word2Vec, FastText. <https://radimrehurek.com/gensim/>
7. **SHAP** — Lundberg, S. & Lee, S.-I. (2017). *A Unified Approach to Interpreting Model Predictions*. NeurIPS.
8. **Friedman, J. H.** (2001). *Greedy function approximation: A gradient boosting machine*. Annals of Statistics.
9. **Breiman, L.** (2001). *Random Forests*. Machine Learning, 45(1), 5-32.
10. McKinney, W. (2017). *Python for Data Analysis*, 2nd Edition, O'Reilly Media.
11. Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*, Cambridge University Press.
12. Géron, A. (2019). *Hands-On Machine Learning*, 2nd Edition, O'Reilly Media.
13. Cahier des charges « Eco-Smart Classifieur », Master 2 — 2025/2026.

18. Annexes — Reproductibilité

18.1 Reproduire en 3 commandes

```
pip install -r requirements.txt
python nettoyage.py           # Pipeline complet (~5 min)
uvicorn main:app --reload     # API + UI sur http://127.0.0.1:8000
```

18.2 Pipeline DVC complet

```
dvc init
dvc repro           # Exécute tous les stages dans l'ordre
```

18.3 Tests + coverage

```
pytest tests/ --cov=. --cov-report=term-missing
```

18.4 MLflow UI

```
mlflow ui           # http://127.0.0.1:5000
```

18.5 Docker

```
docker build -t eco-smart .
docker run -p 8000:8000 eco-smart
```

18.6 Configuration de l'aléatoire

`random_state=42` partout (split, RandomForest, HistGradientBoosting, Word2Vec, FastText, KMeans) pour garantir la reproductibilité.

18.7 Variables d'environnement

Variable	Défaut	Description
MLFLOW_TRACKING_URI	file:./mlruns	Backend MLflow
DISABLE_MLFLOW	0	Si 1, désactive le tracking MLflow
PYTHONIOENCODING	—	Mettre à <code>utf-8</code> sur Windows

18.8 Structure du dépôt

```
ML_Project/
```

— nettoyage.py	# Pipeline d'entraînement
— clustering.py	# Module 3
— benchmark_vectorizers.py	# Module 4
— shap_analysis.py	# Bonus 7
— monitoring.py	# Module 6
— nlp_utils.py	# Tokenizer FR partagé
— main.py	# Backend FastAPI
— index.html	# Frontend 3 onglets
— tests/	# 44 tests pytest
— artifacts/	# Modèles + rapports (générés)
— mlruns/	# MLflow tracking
— .github/workflows/ci.yml	# CI
— dvc.yaml	# Pipeline DVC
— Dockerfile	# Image FastAPI
— DEPLOY.md	# Guide déploiement
— PROMPTS.md	# Journal IA (obligatoire)
— README.md	# Quickstart 3 commandes
— rapport_ML_Projet.md	# Ce rapport

